

uLLM : A Unified API for Simplifying LLM Use in the Classroom

Hiba Eltigani
Tufts University

Abdullah Bin Faisal
Tufts University

Rukhshan Haroon
Tufts University

Fahad R. Dogar
Tufts University

Abstract

The widespread adoption of large language models (LLMs) in recent years has had a significant impact on computer science education, offering valuable functionalities across various applications. However, as new models are released frequently, each with distinct interfaces and capabilities, educators face challenges in providing easy and equitable access to students—particularly in courses focused on the practical application of LLMs.

This paper reports on experiences from three university courses that used a proxy system (uLLM) designed to simplify access to multiple LLMs, offering built-in retrieval-augmented generation and context management through a unified API. By analyzing log data and student surveys, we identified key usage behaviors, the utility of different API features, and students' perceptions of the simplified interface. Our results indicate that students found the API easy to use and applicable across a variety of tasks, regardless of their prior experience in building similar applications or interacting with LLMs. These insights offer educators practical recommendations for enabling simplified access to LLMs across diverse courses and instructional contexts.

CCS Concepts

• **Social and professional topics** → **Computing education**; • **Human-centered computing** → **Human computer interaction (HCI)**.

Keywords

LLMs, Generative AI, CS Education

ACM Reference Format:

Hiba Eltigani, Rukhshan Haroon, Abdullah Bin Faisal, and Fahad R. Dogar. 2025. uLLM : A Unified API for Simplifying LLM Use in the Classroom. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large Language Models (LLMs) are increasingly used as foundational tools in computer science education, such as for automated grading [20], feedback [4], and content generation [8, 10, 13, 24].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2025/06
<https://doi.org/XXXXXXX.XXXXXXX>

More importantly they are used as *backbones* for student-driven course assignments and projects. For example, courses such as UC Berkeley's CS294/194-196 [22] task students with designing agents and applications that programmatically interface (i.e., via APIs) with LLMs.

Supporting assignment- and project-based courses that incorporate LLMs can place a significant infrastructural burden on educators and course staff, who must manage API access keys for different students, ensure fair and equitable usage, maintain subscriptions with multiple LLM providers, and handle associated costs. From the student perspective, two main challenges emerge. First, differences in APIs across LLM providers (e.g., OpenAI, Anthropic, Google) make it difficult for students to experiment with and switch between different models. This experimentation is crucial for assessing LLM capabilities and integrating them effectively into their specific use cases — a skill that is increasingly essential in today's industry landscape [6, 14].

Second, existing LLM APIs typically lack built-in mechanisms for preserving context — such as conversation history or documents — across multi-turn interactions that extend over the duration of a semester. As a result, students may be required to build their own solutions (e.g., custom databases) to maintain conversational continuity and support document Retrieval-Augmented Generation (RAG) [12]. These additional infrastructure demands can hinder experimentation and distract students from higher-level learning objectives, especially in courses where implementing such techniques is not the primary focus.

In this paper, we present an LLM proxy system (uLLM) designed to simplify access to and use of different LLMs by providing a *unified high-level* API. This API abstracts away diverse low-level implementations while also supporting context management features such as maintaining conversation history and RAG.

We report on the experiences of 58 students who used uLLM across *three* courses:

- (1) **Computer Networks**: where students designed and implemented web accessibility features powered by LLMs
- (2) **Multi-Agent Systems**: where students built reasoning agents incorporating LLM-based capabilities
- (3) **LLMs for Social Good**: where students developed chatbots to address various social challenges

We share our analysis on request-level logs, aimed at understanding students' interaction patterns and usage of different features provided by our API. We also conducted a survey study to get students' perceptions of our system. The main outcomes of our study indicate:

- **Effectiveness**: uLLM successfully supported different use-cases

- Ease of use: Students, including beginners with non-CS backgrounds, found it easy to get started and use
- Feature utilization: Students varied in how they used different features, highlighting diverse experimentation patterns

Our experiences can help guide other institutions and educators seeking to enable LLM access while supporting students with limited experience in building applications. Systems similar to uLLM can facilitate early experimentation with LLMs and help students develop proficiency in evaluating and leveraging these models.

2 Related Work

In recent years, researchers in computer science education have explored various applications of large language models (LLMs) to augment learning activities in computer science courses [21]. LLMs can significantly reduce instructor and tutor workload by assisting with tasks such as grading [15], explaining complex content [23], providing feedback on assignments [7], and automatically generating customized educational materials [8, 10, 13, 24]. Additionally, these models can enhance students' productivity [2] and enable exploration of diverse practical applications, fostering deeper understanding [14].

Moreover, familiarity with integrating LLMs into software applications has become a critical skill, aligning with industry demands and enhancing students' market readiness [11, 14]. LLMs can enhance software applications by enabling advanced features such as converting textual content into visual interface elements [5] and interpreting the emotional tone of text messages to enhance communication accessibility [9]. Recognizing this, educators have recently begun collaborating with industry partners to redesign software engineering courses [14]. Concurrently, new educational tools are being developed specifically to facilitate the seamless integration of LLMs into students' software applications [3]. In this context, our work contributes a simple API interface that enables students to incorporate advanced LLM capabilities into their projects.

3 Methods

3.1 uLLM

We designed and implemented uLLM, a proxy system that allows access to multiple off-the-shelf LLMs, as well as support for context management (i.e., conversation history, RAG) through a high-level API. It exposes three methods (see Table 1):

- **generate.** Allows users to send a query along with a system prompt to an LLM, specified via the `model` parameter, and receive a generated response. The optional `session_id` parameter enables users to maintain conversational state across multiple requests; each session ID effectively serves as a separate "chat window" or thread, allowing students to manage different conversations or tasks in parallel while maintaining context isolation. Users may also specify additional parameters such as temperature, the number of prior messages (`lastk`), and the amount of uploaded RAG context (`rag_threshold`, `rag_k`) to consider. These parameters can vary between requests and across sessions, enabling flexible and customized interactions.
- **file_upload.** The `file_upload` method allows users to upload predefined context to a session in the form of plain text or PDF

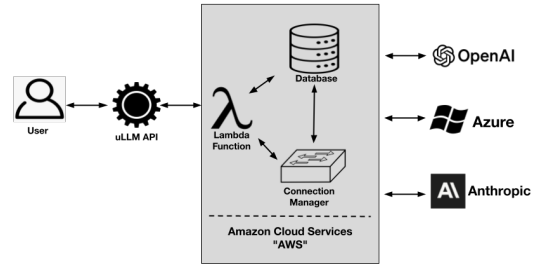


Figure 1: uLLM's Architecture

files. Users can specify whether the proxy should divide the text into fixed-size chunks or apply flexible chunking — preserving semantically similar passages together — by using a strategy parameter. In addition, users can supply metadata to facilitate later retrieval.

- **retrieve.** This method enables users to fetch semantically relevant context chunks for a given query within a session. This capability can support debugging tasks or allow students to provide relevant context to another session ID.

These methods were made available to students through RESTful APIs. While, the proxy is primarily designed for text-based LLM interactions, reflecting the dominant mode of communication across most LLM platforms [6], the design choices support a versatile range of applications and use cases [26]. Finally, requests with missing or non-standard parameters fall back to default, pre-configured values.

Our proxy is implemented on Amazon Web Services (AWS) as a serverless deployment using Lambda functions (Fig. 1 shows general architecture of uLLM). To maintain state — such as conversation history and error tracking — we rely on key-value stores (DynamoDB), while PostgreSQL supports the RAG functionality (e.g., embedding storage, semantic lookup). uLLM is exposed through a RESTful API configured via AWS API Gateway. We also leverage API Gateway features to enforce authorized access and usage limits (described next).

Key Management and Usage Quotas. All requests to uLLM are managed through individual API keys that track usage, enforce quotas, and control access permissions. Preset thresholds per key help prevent abuse, including limits such as a maximum rate of 10 requests per second, 512 requests per day, 10 concurrent requests, a maximum file size of 4MB, and a maximum request duration of 59 seconds. All requests are logged and stored within AWS.

RAG Support. Our system provides RAG capabilities using a combination of Lambda functions and DynamoDB, together with OpenAI's GPT-4. It supports chunking, embedding, and metadata generation, and stores embeddings in a vector database to enable semantic retrieval.

3.2 Study Context

Computer science students at a medium-sized private university in the United States used uLLM in class projects. A total of 58 students from three different courses during the Fall 2024 and Spring 2025

Table 1: Example of different types of uLLM’s Methods

<pre>generate (model = '4o-mini', system = "Explain like I'm five", query = "What is computing?", temperature = 0.0, lastk = 0, session_id = 'GenericSession', rag_threshold = 0.5, rag_k = 1)</pre>	<pre>file_upload (path = "path/to/your/document.pdf", strategy = 'smart', description = 'details about ABC', session_id = 'GenericSession')</pre>	<pre>retrieve (query = "Tell me about ACM?", session_id = 'GenericSession', rag_threshold = 0.5, rag_k = 1)</pre>
--	---	---

semesters incorporated uLLM into various assignments. The three classes had distinct focuses.

- **Course-A.** A networking course (NW), in which students built an LLM-powered HTTPS proxy as their final group project. uLLM was used to enhance the performance of the proxy—some implementations improved accessibility features, while others focused on reducing page load times. Students in this class interacted with an earlier version of uLLM that only supported the `generate` method.
- **Course-B.** Focused on building multi-agent systems (MAS). In the final project, students developed and evaluated a hybrid system combining traditional agents with LLM-based ones. These systems were applied in domains such as path-finding, voting, auctions, and game theory. Students utilized LLMs to simulate various roles within these domains.
- **Course-C.** LLMs for Social Good (LSG) focused exclusively on building LLM-based applications. Students used the proxy throughout the semester to develop multiple projects centered around chatbot systems that address social challenges faced by marginalized communities. During the first six weeks of the semester, students completed individual projects aimed at understanding various aspects of LLMs, such as prompt engineering and RAG usage, before progressing to more advanced group projects incorporating techniques like agentic workflows and human-in-the-loop design. These applications covered a wide range of topics, including university advising, music recommendation, news aggregation, study planning, course tutoring, accessibility benchmarking, and ethical evaluation.

The respective instructors of the courses introduced uLLM to students. Additionally, students were provided with documentation and example-based guides. A custom GPT-powered assistant [16] trained on this documentation was also made available. Students in all courses were allowed to use a set of six models: OpenAI GPT-4o-mini [1], Anthropic Haiku-3 [18], Azure Phi-3 [19], and Meta LLaMA 3 models in 1B, 3B, and 8B variants [17].

3.3 Data Analysis

We analyzed the log data from all three courses to understand students’ usage of uLLM. Our analysis focuses on structurally correct requests received uLLM, regardless of whether they were executed successfully. In addition, we conducted a survey for students in Course-C at the end of the semester to understand their perceptions

of uLLM and how their prior experience may have influenced their interaction with the API. Students in the course had the longest period of interaction with uLLM and used it in a variety of use cases. Of the 29 students enrolled in the course, only 8 fully completed the survey.

Ethical Considerations. The study was approved by our Institutional Review Board (IRB), and the survey was launched anonymously. No personally identifiable information was collected, and survey responses could not be linked to individual participants.

4 Results

4.1 uLLM’s Methods Usage

The 58 students sent a total of approximately 75K requests over 145 days, averaging 500 requests per day. The `generate` method was the most frequently used, accounting for 83.6% of all requests, followed by the `file_upload` and `retrieve` methods at 12.8% and 3.6%, respectively. Approximately 14% of the `generate` requests used RAG. When examining distinct user behavior, a similar pattern emerges as with the overall usage: most users employed the `generate` method alongside other functions, while approximately 50% used it exclusively. Notably, no users relied solely on the `file_upload` or `retrieve` methods.

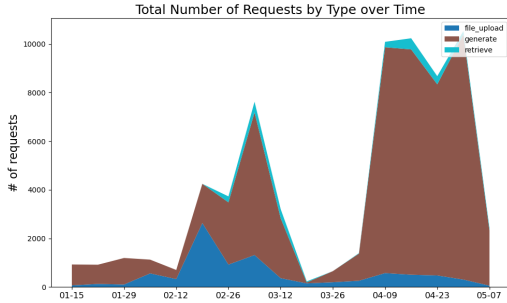
Almost 90% of the documents added for RAG purposes were PDFs, with only one student using text files exclusively to provide context. The choice of media type appears to be influenced by the type of applications developed: students in Course-B used text files exclusively, possibly indicating user-generated content. In contrast, students in Course-C relied heavily on readily available materials to enhance response quality and reduce hallucinations.

Students across the various courses also differed in their utilization of the available methods. While students in Course-A had access only to the `generate` method, those in Course-B did not use the `retrieve` method at all, whereas Course-C students used it in 3% of their requests. OpenAI’s GPT-4o-mini was the most used LLM across all courses, though the rankings of the remaining models varied.

We also observed a small number of requests with empty model fields, unsupported models (e.g., `gemin-2.0-flash-001`), and fabricated model names (e.g., `bad-model`), which might indicate user testing or input errors. An analysis of cost showed that total request charges were under \$10; however, the overhead associated with RAG support (including metadata generation, chunking, and

Table 2: Usage of uLLM by Course

	Course-A (NW)	Course-B (MAS)	Course-C (LSG)
# of students	18	11	29
# of requests	2,003	2,964	70,117
Requests per function(%):			
generate	100	97	83
file_upload	0	3	14
retrieve	0	0	3
# of call requests w/ RAG	0	322	8765
Top 3 models used:			
1st	GPT4o-mini (93%)	GPT4o-mini (89%)	GPT4o-mini (72%)
2nd	Anthropic Haiku3 (4%)	Azure Phi3 (9%)	Meta llama 3B (14%)
3rd	-	Anthropic Haiku3 (1%)	Anthropic Haiku3 (13%)
Context Added by file type(%):			
Text Files	NA	100	90
PDF Files	NA	0	10
Total request cost (\$)	2.3	0.3	8.9
Aggregate RAG Overhead Cost (all courses)		63.8	

**Figure 2: Course-C Students' Usage over Time**

embedding creation) was approximately six times the total request cost. Additional details are shown in Table 2.

4.2 Course-C Use Case

We examined how students in Course-C utilized the various parameters of uLLM and whether their behavior changed over the course of the semester. Our results indicate that most activity occurred during group projects, with a peak during the final project at the end of the semester (see Fig. 2).

The generate, file_upload, and retrieve methods were used by 29, 27, and 8 users, respectively. While the usage of the first two methods was consistent, the retrieve method was used sporadically, and no student demonstrated consistent use of it.

Models. Despite the first assignment requiring students to try three different models, 71% of students used only GPT-4o-mini for all their requests. Anthropic Haiku was used by 7 students, Azure Phi3 by 3 students at various points in time, and the Meta LLaMA 3B model by one student who was benchmarking model

performance on a text evaluation task. This benchmarking activity led to a substantial increase in requests for both Anthropic Haiku and Meta llama models (Fig. 3(a)).

Session ID. This parameter is the primary mechanism for context management. Students used it in varied ways: to simulate interactions from new users, enable simultaneous multi-user communication, or separate roles within a system. Consequently, it showed the highest variability – only six students used fewer than five IDs, while eight used more than 100.

Students showed different attitudes toward exploring parameter values. For example, rag_threshold values varied widely throughout the semester (Fig. 3(b)), while the chunking strategy parameter saw limited exploration early on, before most students adopted the smart strategy. Toward the end of the semester, rag_threshold values trended lower, which might suggest that students either gained more experience in selecting relevant context files to upload or developed trust in the model's ability to identify relevant text. Table 3 provides additional details on parameter usage.

4.3 Students Perceptions of uLLM

The student survey was completed by 28% of students in Course-C. A majority of respondents (75%) found uLLM easy to get started with, and 50% found it easy to integrate into their workflow. Views on debugging were mixed: 25% reported it was difficult to identify and fix issues, while another 25% found it easy. Most respondents (63%) identified themselves as beginners in their prior experience with APIs and incorporating LLMs into applications.

Half of the respondents perceived their experience with uLLM better than similar APIs, with 10% describing it as significantly better. Those who reported a worse experience referred to issues encountered with earlier versions of uLLM, as reflected in one comment, "Initial API was poorly constructed with limited features;

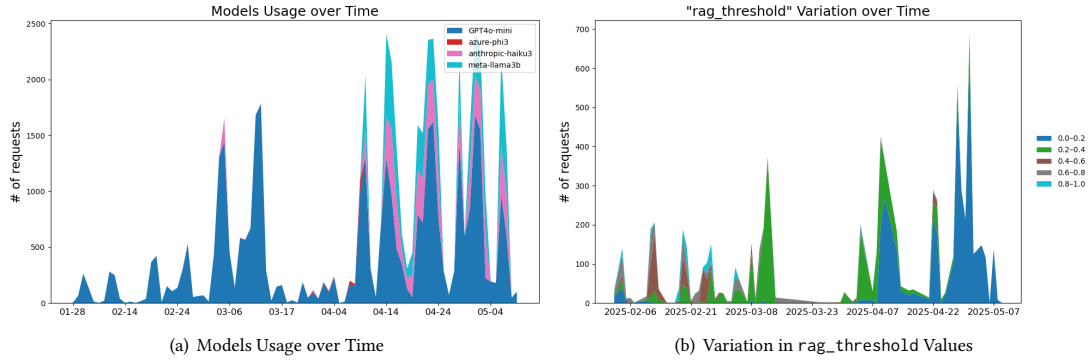


Figure 3: Details of user activity and usage patterns

Table 3: Usage of uLLM Parameters by Students in LSG Course

Parameter	Usage	Requests Using Example Values from the Documentation (%)	Students with fixed value in all requests (%)	Highest number of unique values used by one student	Temporal pattern of value changes
temperature	Controls response randomness: higher values produce more unpredictable responses; lower values yield more deterministic ones	58%	25%	10	Consistent exploration of values over the semester
lastk	Controls how many previous query-response pairs are included in the current request	67%	14%	22	Consistent exploration of values over the semester
rag_k	Controls how many relevant chunks from pre-stored files are included in the current request	11%	26%	9	High early exploration followed by later stabilization
rag_threshold	Sets the similarity threshold for including file chunks: higher values require closer relevance to the query	5%	22%	12	Consistent exploration of values over the semester
strategy	Controls whether file chunking uses an LLM for semantic grouping or a fixed chunk size	NA (all strategies were recommended for different files)	59%	2 (Max available)	Lower early exploration followed by later fixed value

after a few months, it became better and the features improved.” All respondents agreed that uLLM supported most of their use cases, although half noted the need for some workarounds.

Respondents generally found the API features useful (Fig. 4), with model switching and conversation history receiving the most positive feedback. However, some features were underutilized: 12.5% of

respondents did not use RAG, and 25% did not use multiple sessions for context management.

Several respondents highlighted the need for more documentation. One explained, “Features like text retrieval worked well for me. I’d love to have detailed documentation on the response format, so I can quickly reference it instead of manually printing out the

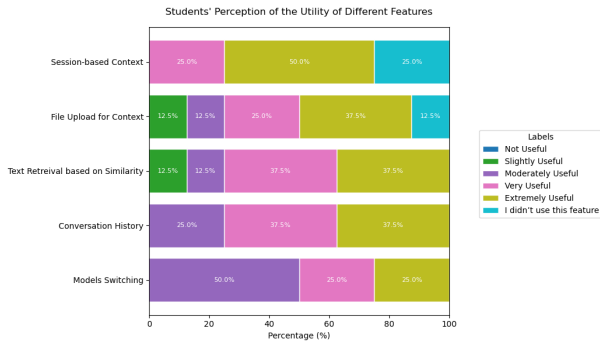


Figure 4: Students' Perception of uLLM's Features

response to check the structure of the returned JSON object.” Another added, “Maybe some better documentation of how the different features integrate together.” In addition, some respondents reported access issues, which echoed similar feedback from Course-B students who mentioned API latency and request limits as areas for improvement.

5 Discussion

Initial results demonstrated the potential of using a system like uLLM to facilitate access to LLMs across diverse educational use cases. Our analysis highlights the need for further research into students' mental models when interacting with LLMs. For example, it remains unclear whether students' heavy reliance on GPT-4o-mini was driven by example code in the documentation or by their perception of that model family's superiority. Additionally, many provided parameters were underutilized— a sizeable fraction of students consistently used a single value across all requests. Understanding these patterns could help bridge current gaps in CS students' preparation for working with AI [14], and we view uLLM as a valuable first step in that direction.

Our analysis also revealed the significant overhead associated with supporting RAG; future optimization is needed to ensure scalability across more courses. Moreover, understanding the cost implications of LLM use is itself a valuable educational outcome [3], and future versions of uLLM could expose these aspects more explicitly. Additional enhancements—such as improved documentation, clearer error messages, and tools for instructors to monitor student usage—would further improve the system's effectiveness and broaden its applicability.

5.1 Limitations and Future Directions.

Despite the limited number of survey respondents, our study provides initial insights into the utility of systems that facilitate students' access to LLMs. However, our findings focus solely on the student perspective and do not capture the viewpoints of instructors. This is especially important, as instructors are often the ones addressing student issues [25]. Their insights can also guide the configuration of usage limits and parameter settings tailored to specific course needs.

For future work, a larger survey conducted across multiple courses can provide comprehensive results. Including instructor perspectives would further inform the design and operational decisions behind such systems, ensuring they are better aligned with instructional goals and classroom dynamics.

6 Conclusion

In this paper, we shared our experience designing and deploying a proxy system to facilitate students' access to multiple LLMs through a unified interface. The system was used by 58 students across three different courses. Our findings suggest that the system is a viable tool for students with varying levels of experience with LLMs and APIs, as demonstrated in the survey responses. Our results highlight the need for further exploration of students' mental models when interacting with LLMs in educational settings, which could inform the design of future tools and educational systems.

Acknowledgments

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Isaac Alpizar-Chacon and Hieke Keuning. 2025. Student's Use of Generative AI as a Support Tool in an Advanced Web Development Course. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 312–318.
- [3] Timothy J Aveni, James Smith, Armando Fox, and Björn Hartmann. 2025. Supporting Students in Prototyping AI-backed Software with Hosted Prompt Template APIs. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 65–71.
- [4] Imen Azaiz, Natalie Kiesler, and Sven Strickroth. 2024. Feedback-generation for programming exercises with gpt-4. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. 31–37.
- [5] Yoonseo Choi, Eun Jeong Kang, Seulgi Choi, Min Kyung Lee, and Juho Kim. 2025. Proxona: Supporting Creators' Sensemaking and Ideation with LLM-Powered Audience Personas. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–32.
- [6] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A Becker, and Brent N Reeves. 2024. Prompt Problems: A new programming exercise for the generative AI era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 296–302.
- [7] Paul Denny, Stephen MacNeil, Jaromir Savelka, Leo Porter, and Andrew Luxton-Reilly. 2024. Desirable characteristics for ai teaching assistants in programming education. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. 408–414.
- [8] Avid Fayaz, Richard Glassey, and Alexander Baltatzis. 2025. Generating Personalized Assignments with Students in the Loop. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 305–311.
- [9] Rukhshan Haroon and Fahad Dogar. 2024. Twips: A large language model powered texting application to simplify conversational nuances for autistic users. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*. 1–18.
- [10] Mollie Jordan, Kevin Ly, and Adalbert Gerald Soosai Raj. 2024. Need a programming exercise generated in your native language? chatgpt's got your back: Automatic generation of non-english programming exercises using openai gpt-3.5. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 618–624.
- [11] Vassilka D Kirova, Cyril S Ku, Joseph R Laracy, and Thomas J Marlowe. 2024. Software engineering education must adapt and evolve for an llm environment. In

- Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1.* 666–672.
- [12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. (2021). arXiv:2005.11401 <https://arxiv.org/abs/2005.11401>
 - [13] Evanfiya Logacheva, Arto Hellas, James Prather, Sami Sarsa, and Juho Leinonen. 2024. Evaluating contextually personalized programming exercises created with generative AI. In *Proceedings of the 2024 ACM Conference on International Computing Education Research-Volume 1*. 95–113.
 - [14] Daniel Mejia, Ernest DV Holmes, Jenn Marroquin, and Jamie Gorson Benario. 2025. Bridging Academia and Industry: Leveraging Generative AI in a Software Engineering Course for Practical Industry Experiences. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 507–513.
 - [15] Goda Nagakalyani, Saurav Chaudhary, Varsha Apte, Ganesh Ramakrishnan, and Srikanth Tamilselvam. 2025. Design and Evaluation of an AI-Assisted Grading Tool for Introductory Programming Assignments: An Experience Report. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 805–811.
 - [16] n.d. [n. d.]. <https://help.openai.com/en/articles/8554397-creating-a-gpt/>.
 - [17] n.d. 2025. Llama 3 models. <https://www.llama.com/models/llama-3/>.
 - [18] n.d. 2025. Meet Claude Anthropic. <https://www.anthropic.com/claude>.
 - [19] n.d. 2025. Phi open model family. <https://azure.microsoft.com/en-us/products/phi/>.
 - [20] Ha Nguyen and Vicki Allan. 2024. Using GPT-4 to provide tiered, formative code feedback. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 958–964.
 - [21] Nishat Raihan, Mohammed Latif Siddiq, Joanna CS Santos, and Marcos Zampieri. 2025. Large language models in computer science education: A systematic literature review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 938–944.
 - [22] UC Berkeley Robotics and AI Research. 2024. CS294/194-196: Large Language Model Agents. <https://rdi.berkeley.edu/llm-agents/f24>. Accessed: 2025-07-03.
 - [23] Liam Stienstra, Abdallah Mohamed, and Mostafa Mohamed. 2025. Exploring GenAI as a Tutoring Tool: A Case Study in First-Year Computer Programming. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 452–457.
 - [24] Ismael Villegas Molina, Audria Montalvo, Shera Zhong, Mollie Jordan, and Adalbert Gerald Soosai Raj. 2024. Generation and Evaluation of a Culturally-Relevant CS1 Textbook for Latines using Large Language Models. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. 325–331.
 - [25] Kevin Shukang Wang and Ramon Lawrence. 2025. Quantitative Evaluation of Using Large Language Models and Retrieval-Augmented Generation in Computer Science Education. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 1183–1189.
 - [26] Zezhu Yu, Suqing Liu, Paul Denny, Andreas Bergen, and Michael Liut. 2025. Integrating small language models with retrieval-augmented generation in computing education: Key takeaways, setup, and practical insights. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 1302–1308.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009